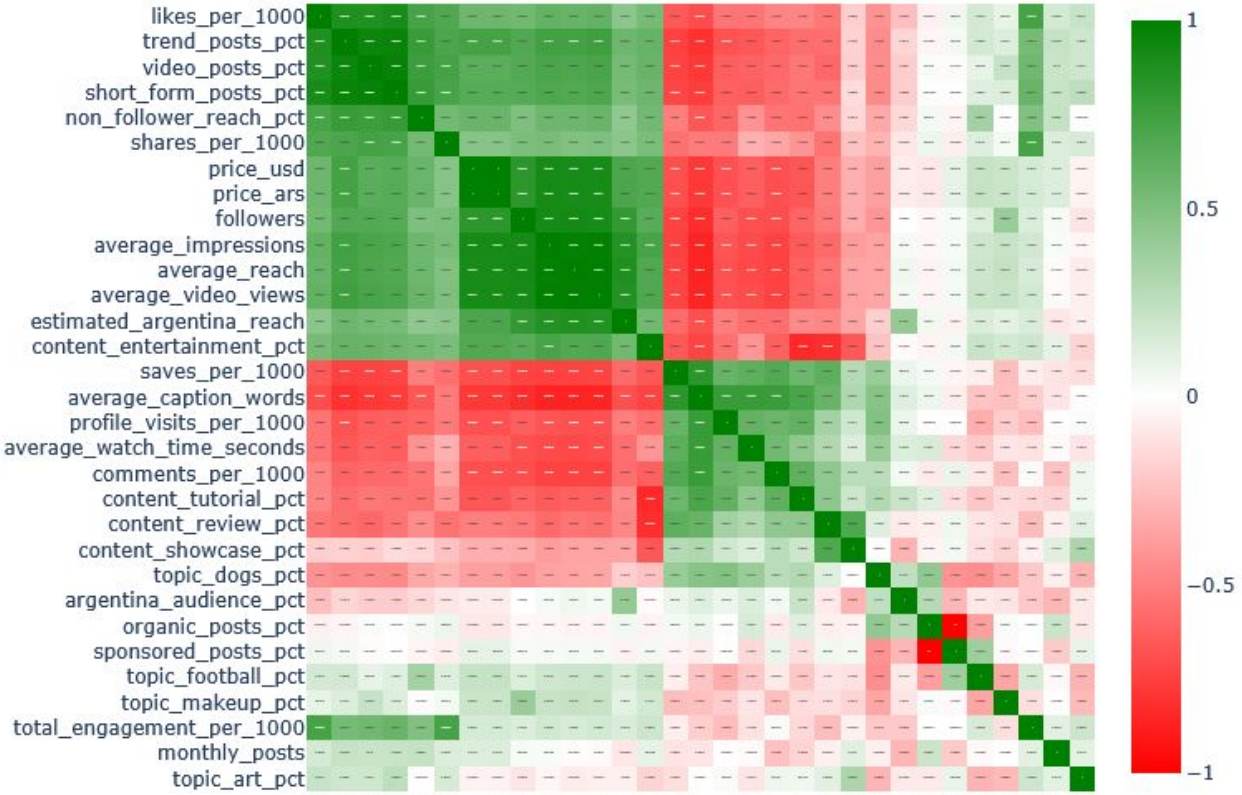
TP4 – Métodos de Aprendizaje NO Supervisado

Lautaro **BONSEÑOR** | Nicolás **RIVAS**
Augusto Andrés **LANARI** | Javier Emmanuel **LIU** |
Juan Ignacio **CAUSSE**

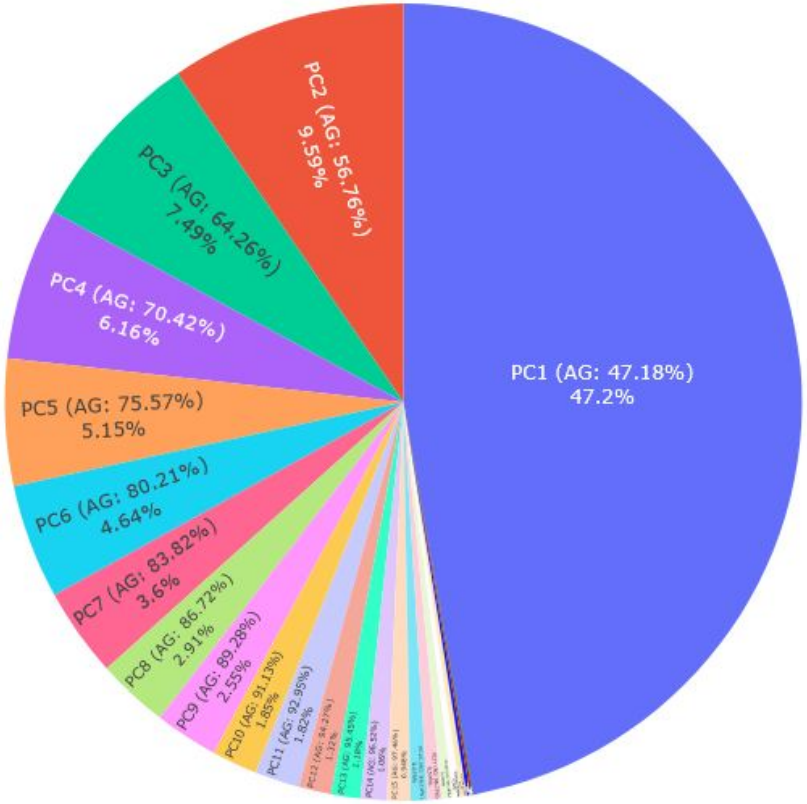
EJ1. Campañas de marketing

Exploración y representación

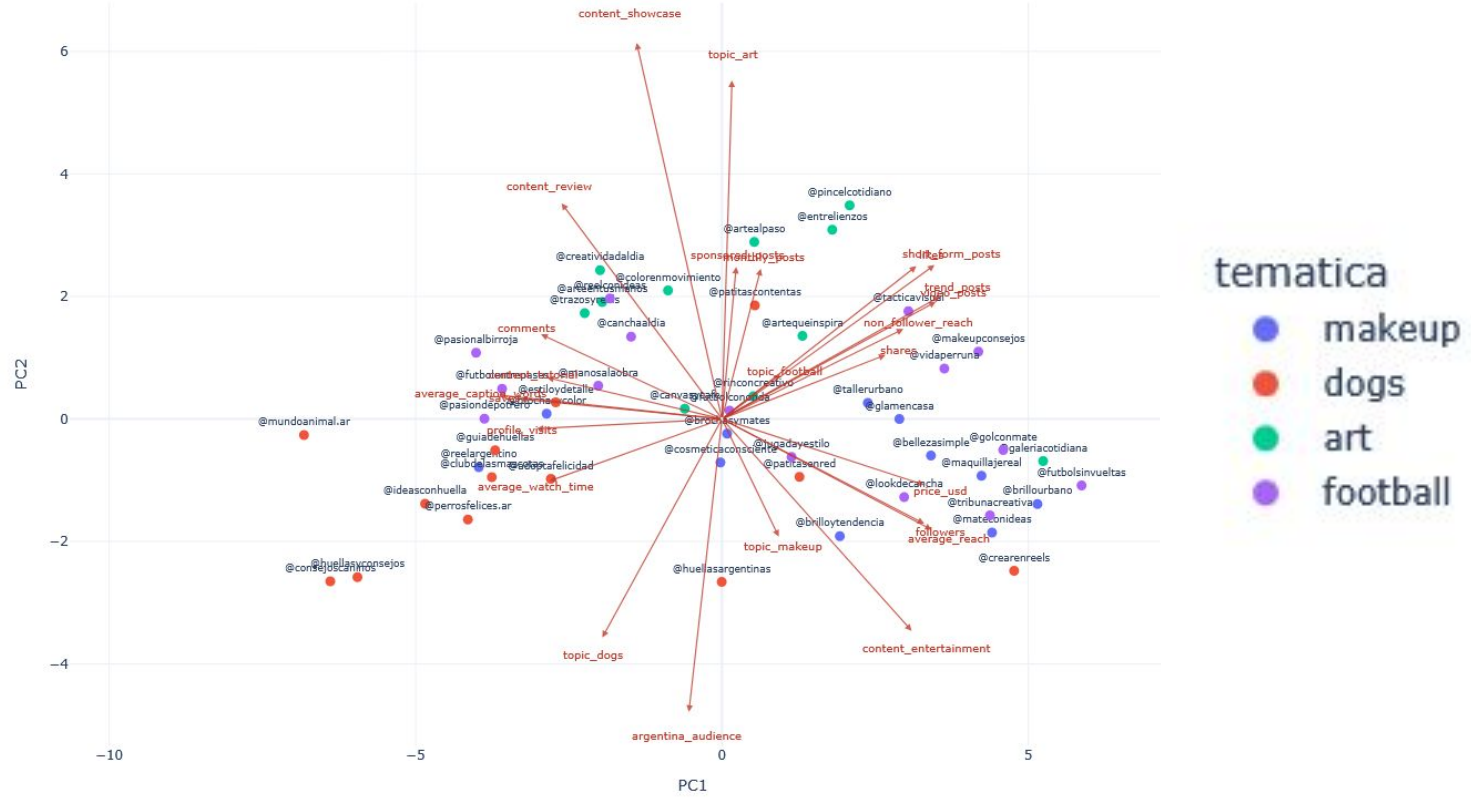
EDA - Matriz de Correlación



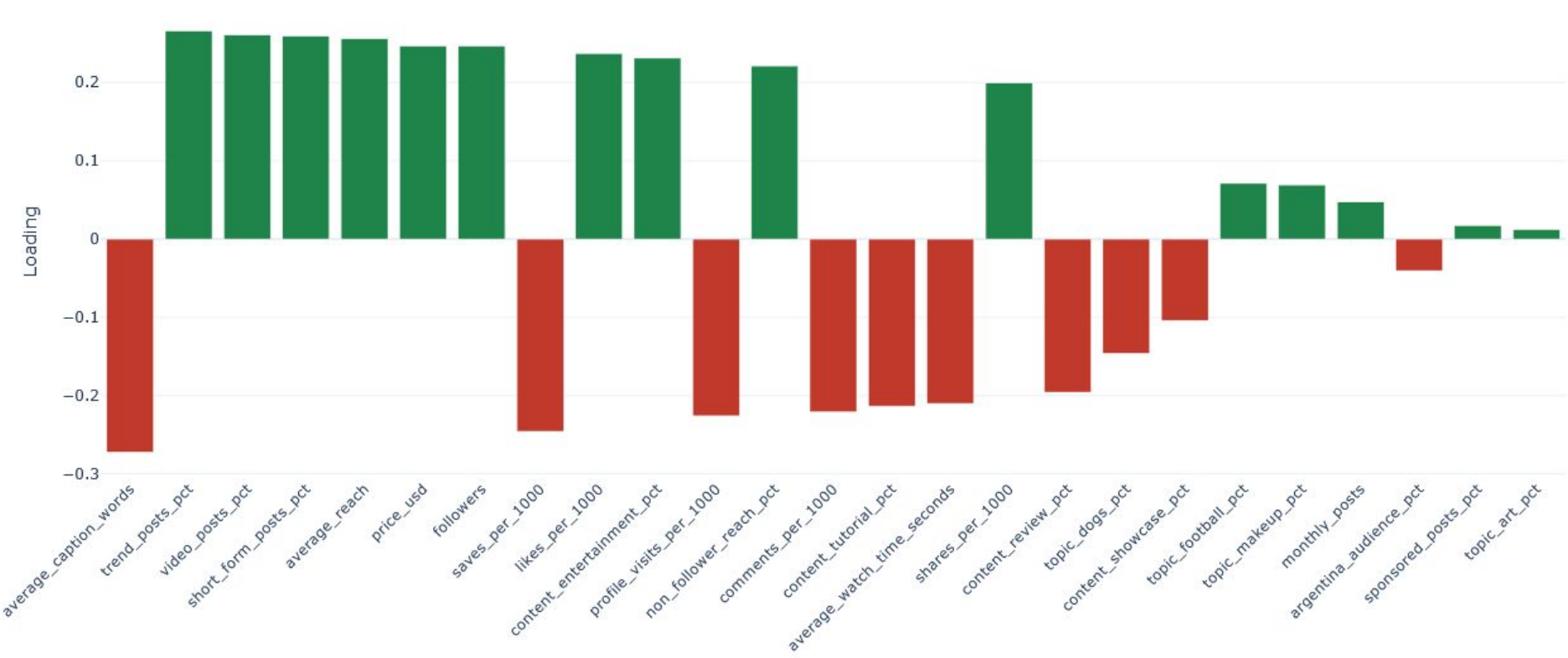
Proporción de Varianza



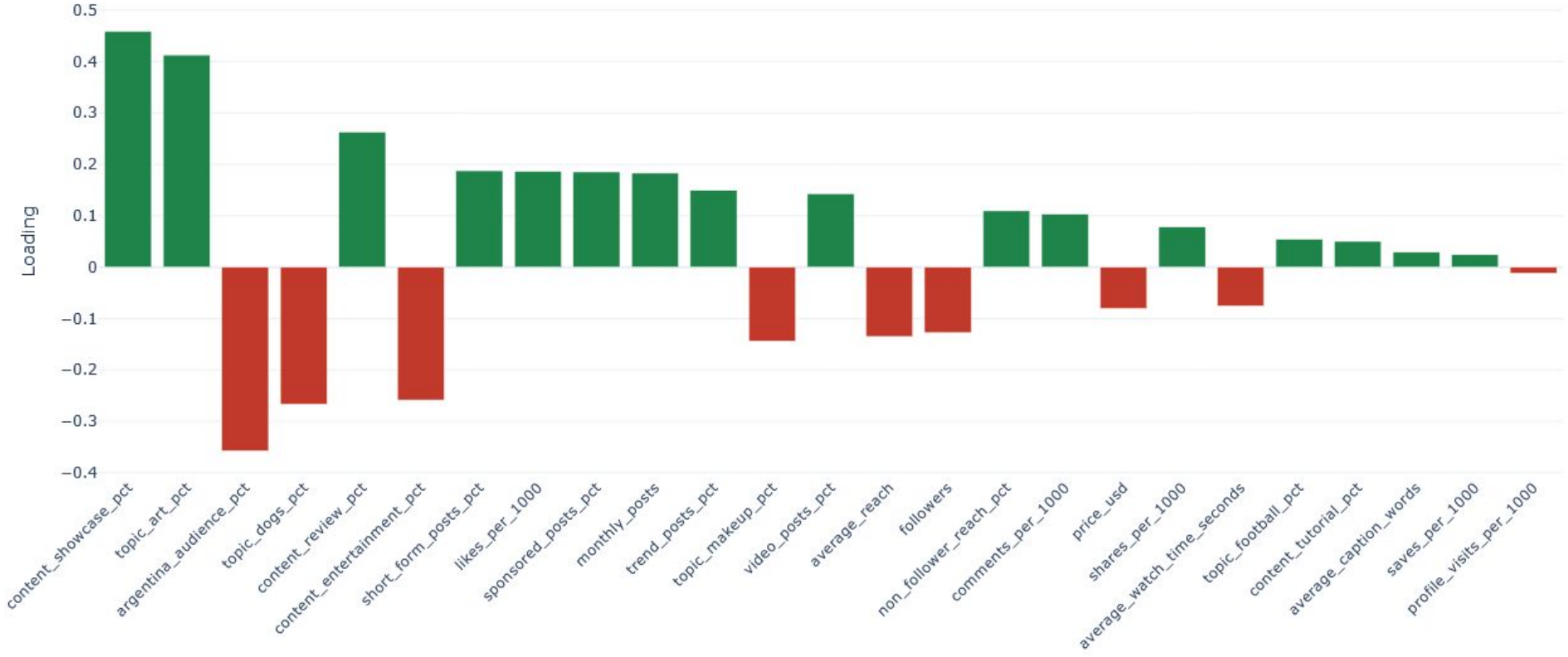
Biplot PC1 (47.2%) vs PC2 (09.6%)



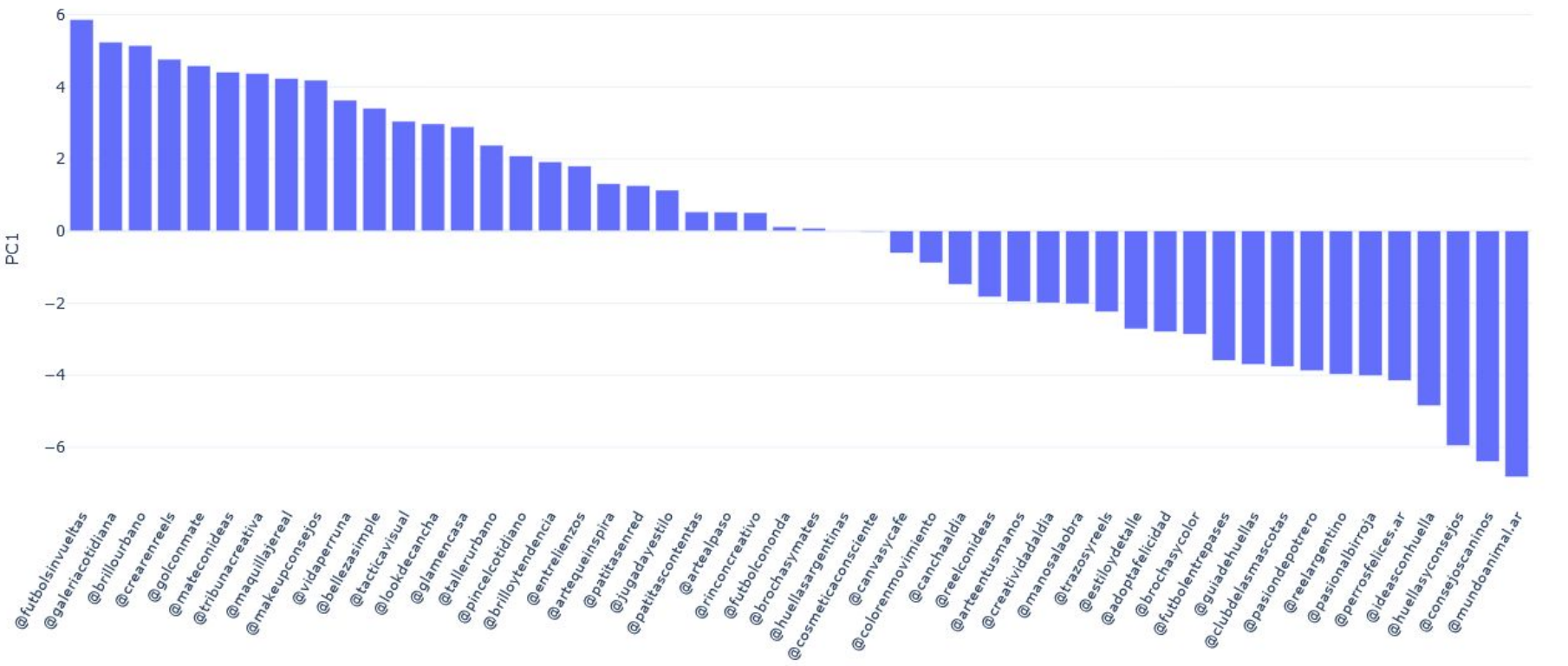
Loadings PC1



Loadings PC2



Índice PC1


PC1
6
4
2
0
−2
−4
−6
@futbolsinvueltas
@galeriacotidiana
@brillourbano
@crearenreels
@golconmate
@mateconideas
@tribunacreativa
@maquillajereal
@makeupconsejos
@vidaperruna
@bellezasimple
@tacticavisual
@lookdecancha
@glamencasa
@tallerurbano
@pincelcotidiano
@brilloytendencia
@entrelienzos
@artequeinspira
@patitasenred
@jugadayestilo
@patitascontentas
@artealpaso
@rinconcreativo
@futbolcononda
@brochasymates
@huellasargentinas
@cosmeticaconsciente
@canvasycafe
@colorenmovimiento
@canchaaldia
@reelconideas
@arteentusmanos
@creatividadaldia
@manosalaobra
@trazosyreels
@estiloydetalle
@adoptafelicidad
@brochasycolor
@futbolentrepases
@guiadehuellas
@clubdelasmascotas
@pasiondepotrero
@reelargentino
@pasionalbirroja
@perrosfelices.ar
@ideasconhuella
@huellasyconsejos
@consejosycaninos
@mundoanimal.ar

Implementación
Redes de Kohonen

Selección y ajuste de hiperparámetros

Matriz 5x5

5000 **iteraciones**

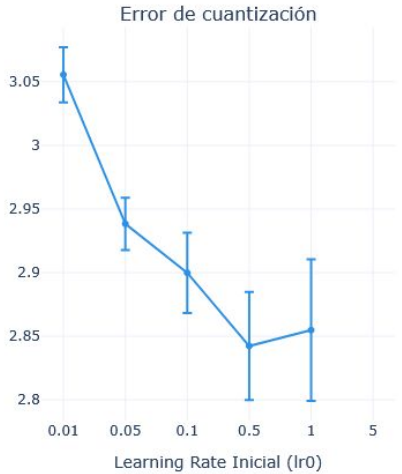
$\mathbf{lr_i}$ – 0.5

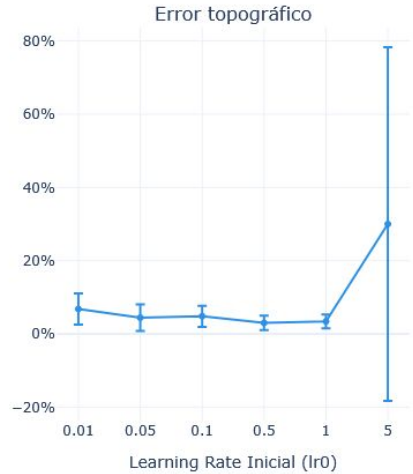
$\mathbf{lr_f}$ – 0.001

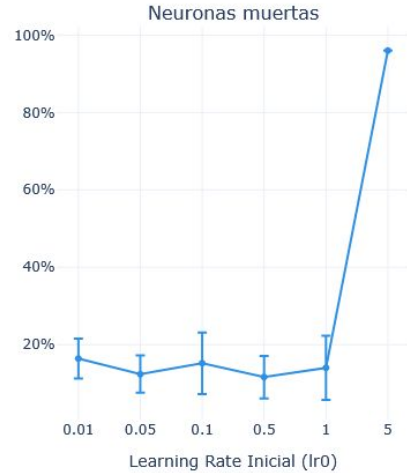
$\mathbf{r_i}$ – 2.5

$\mathbf{r_f}$ – 1.0

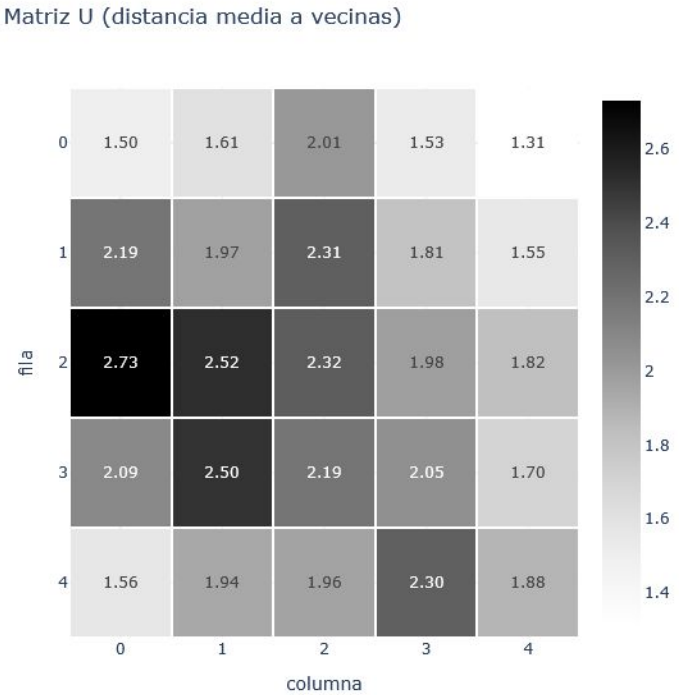
Barrido de Learning Rate inicial

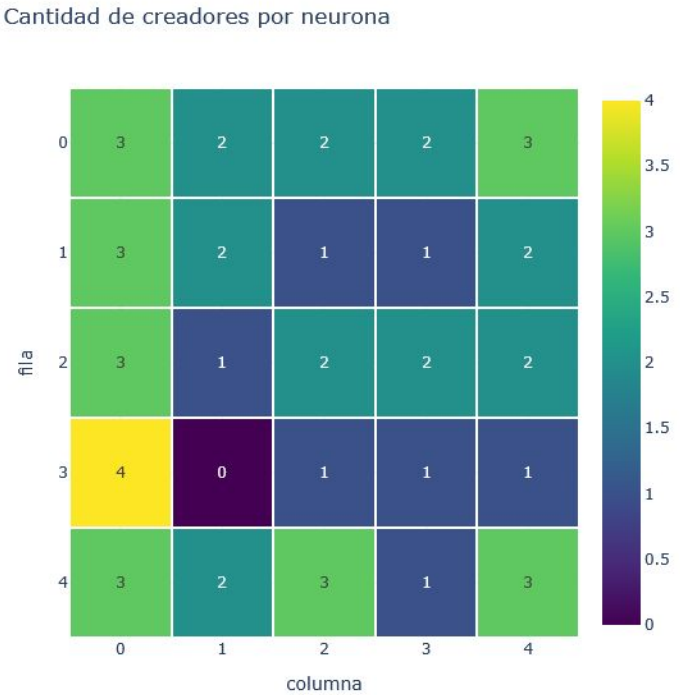




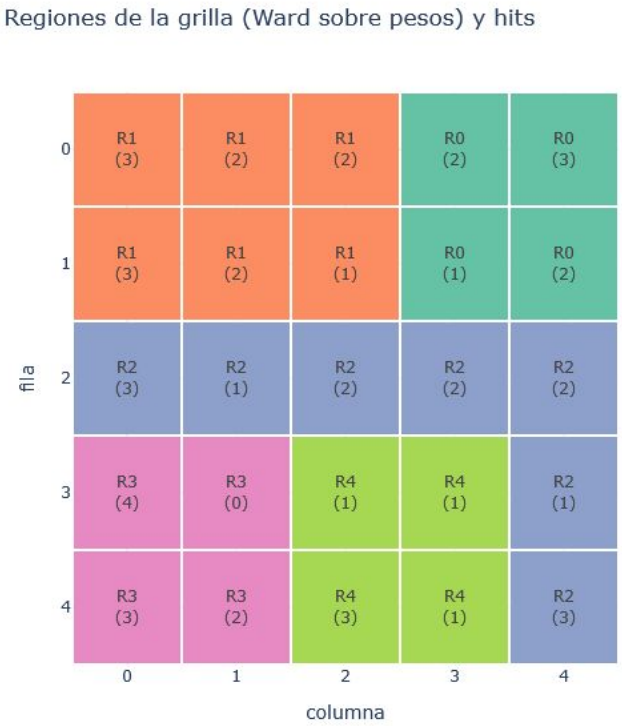


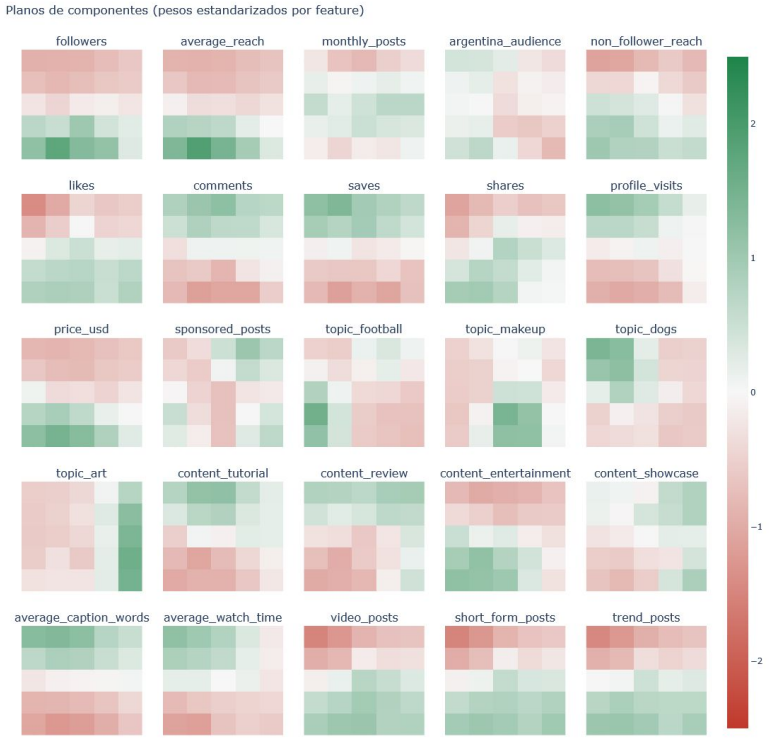
Topología de la red y fronteras



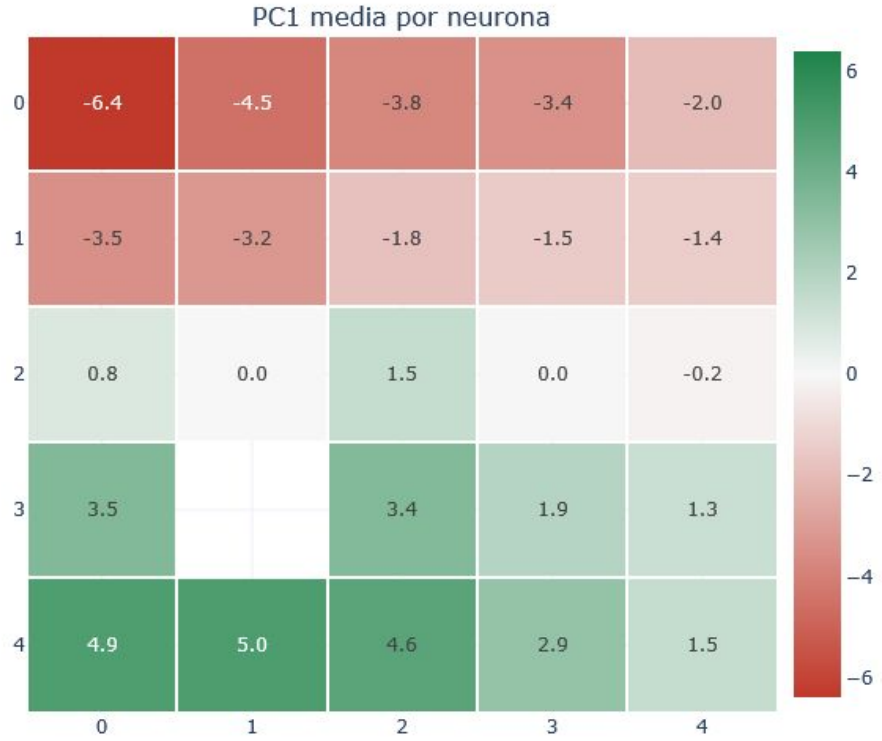


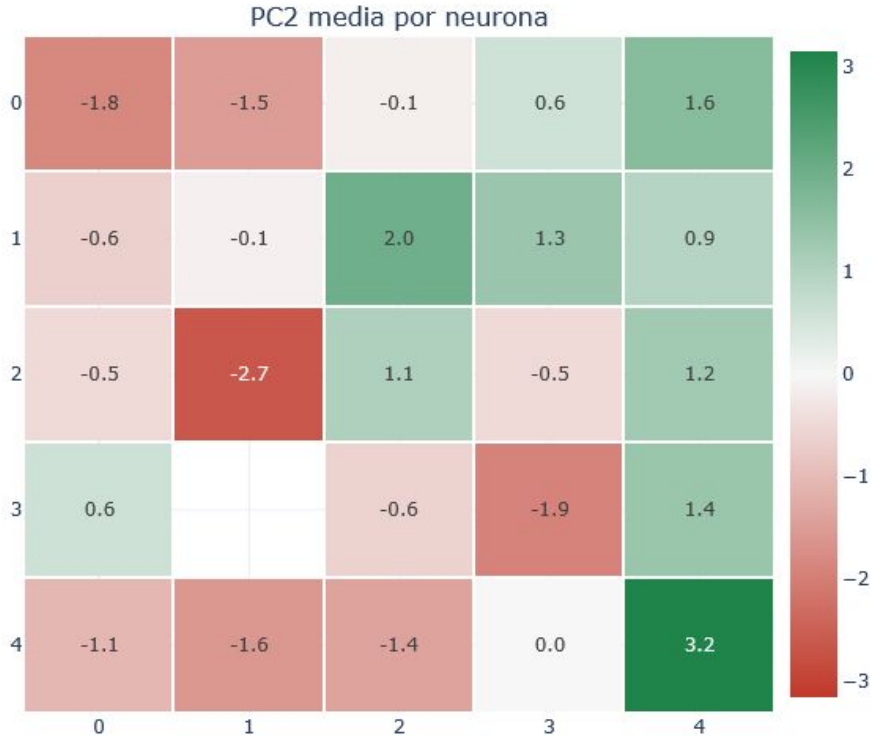
Planos de componentes y regiones de la grilla





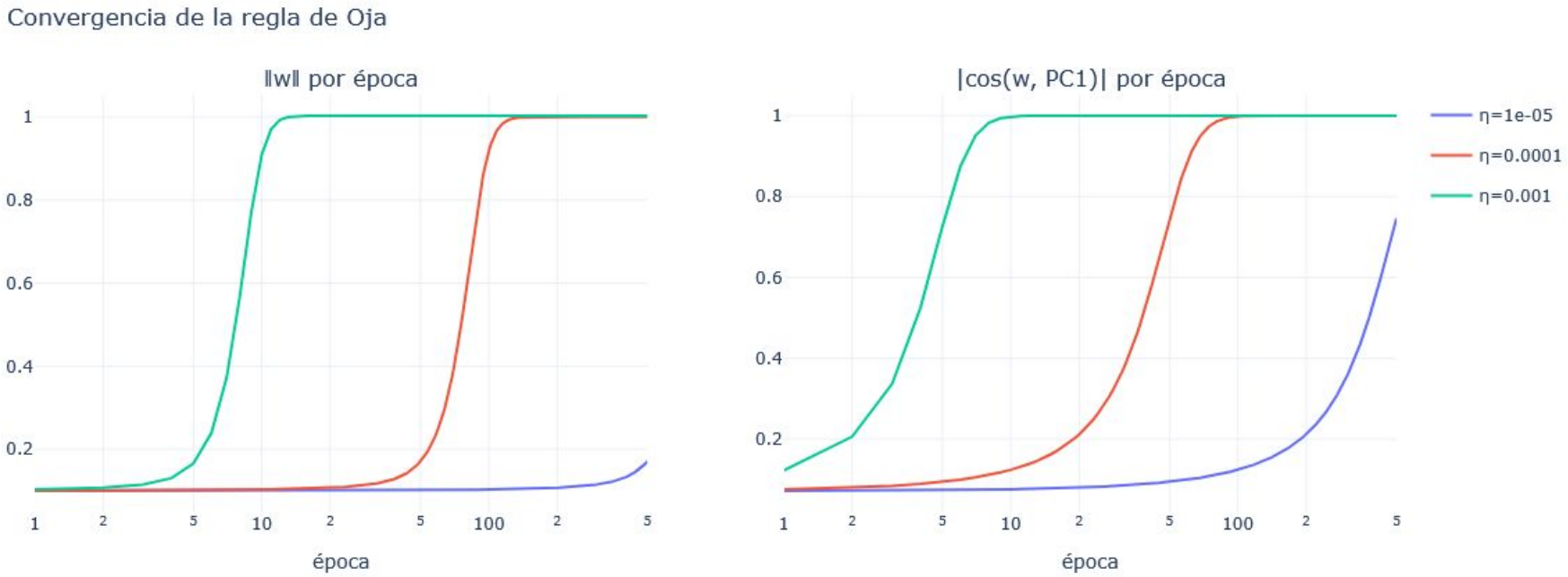
Interpretación de perfiles



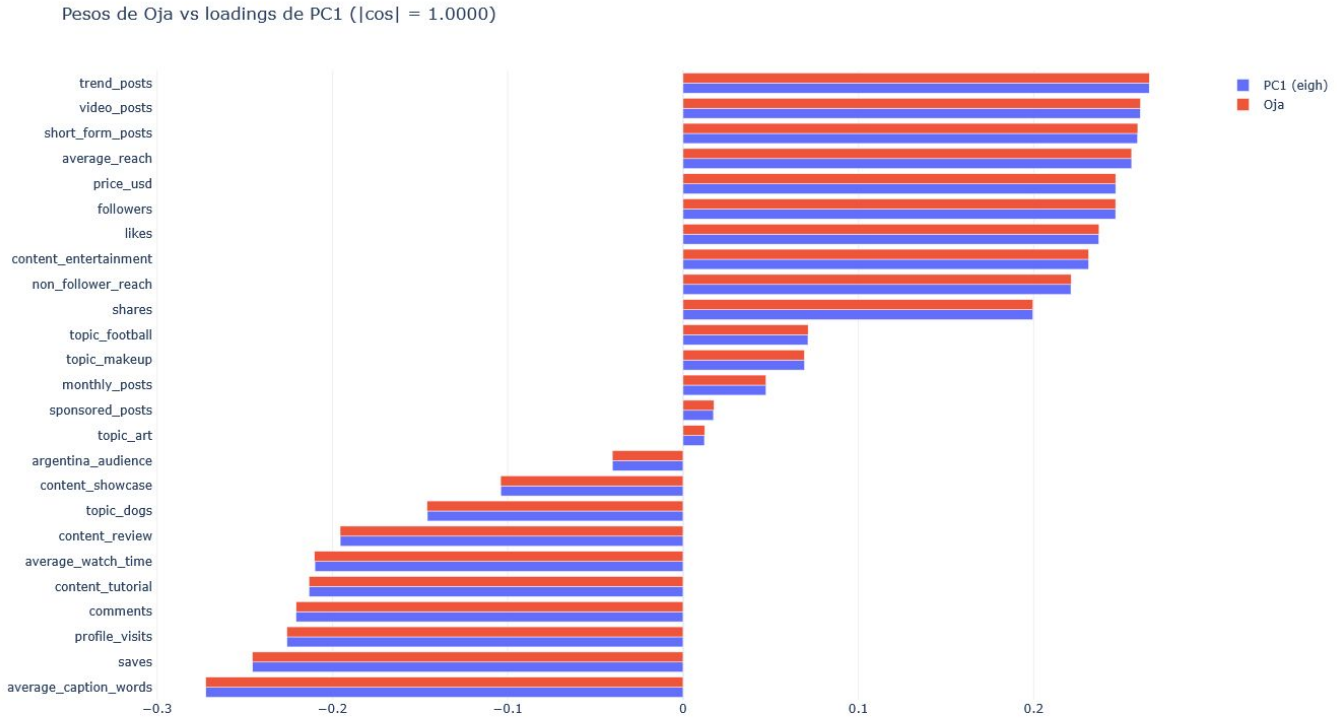


Índice

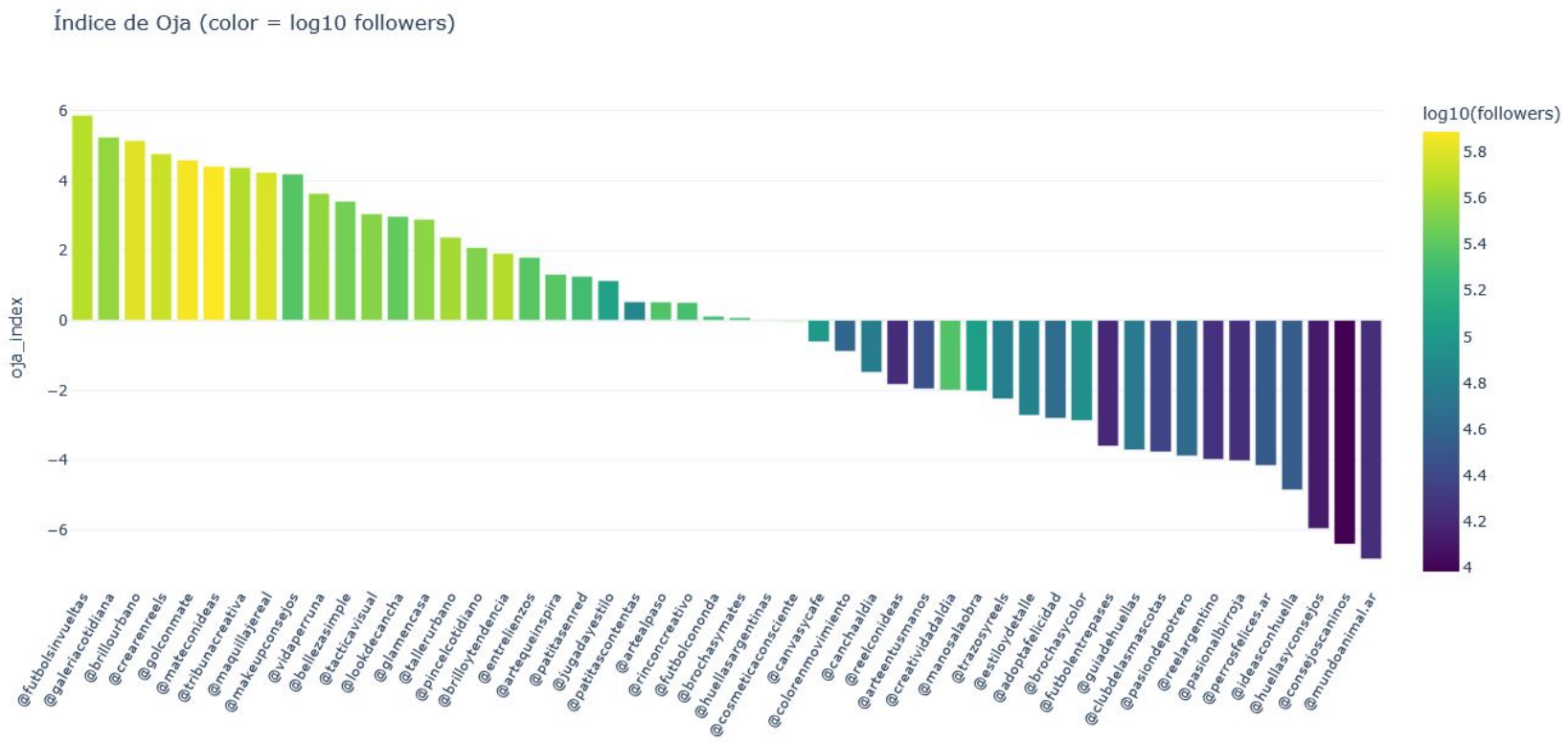
Dinámica de convergencia del modelo (lr=0.0001)



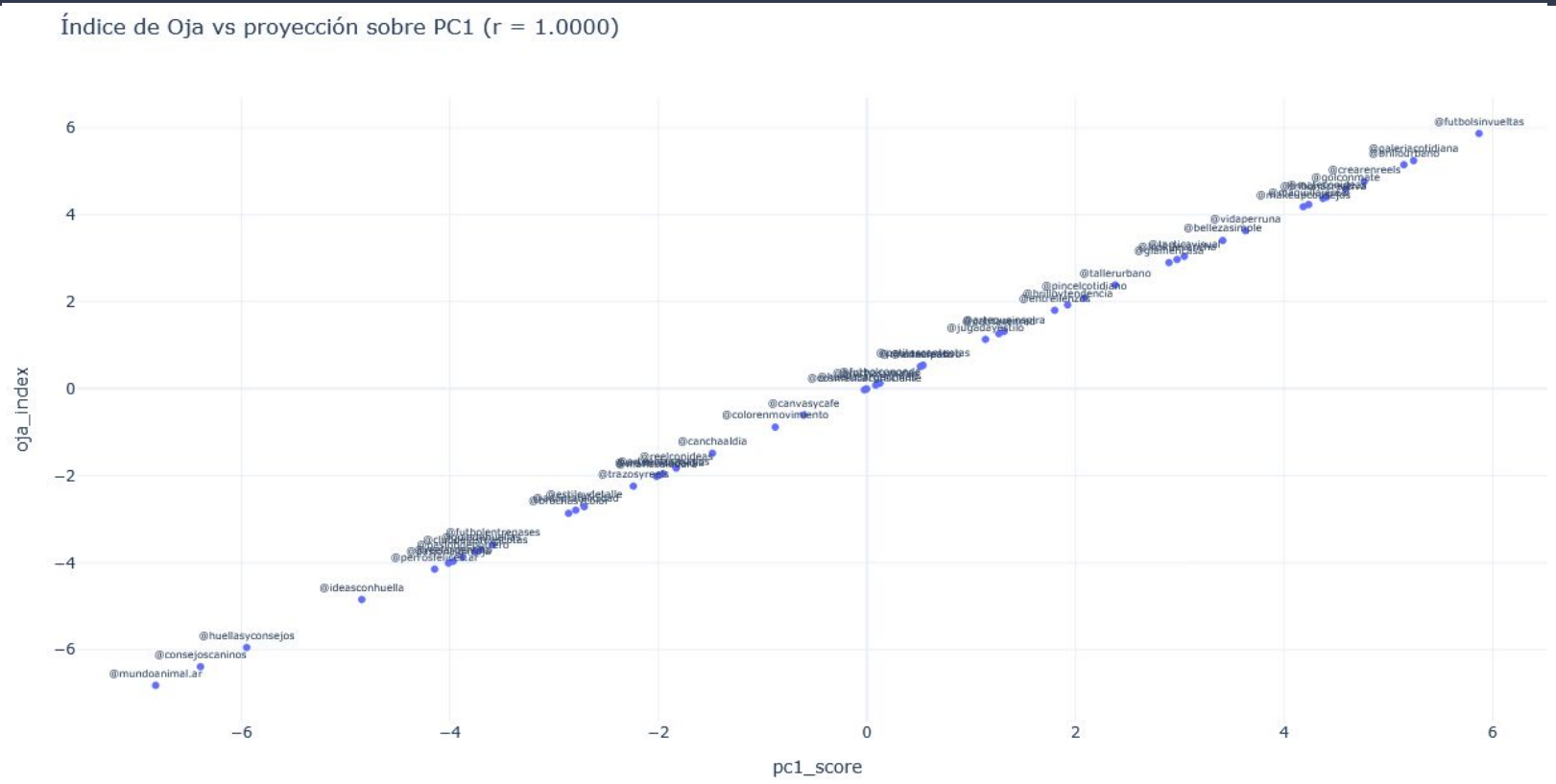
Evolución de los pesos sinápticos vs PC1



Distribución del índice

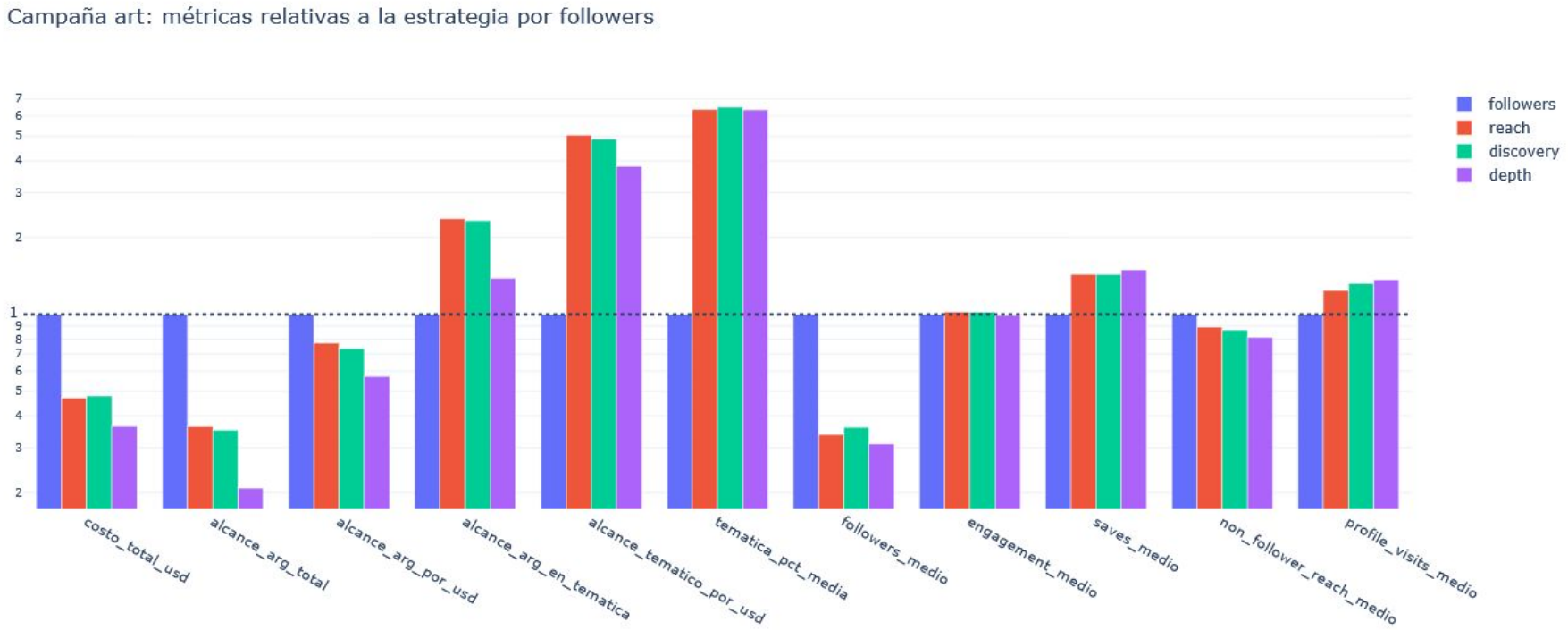


Comparativa directa

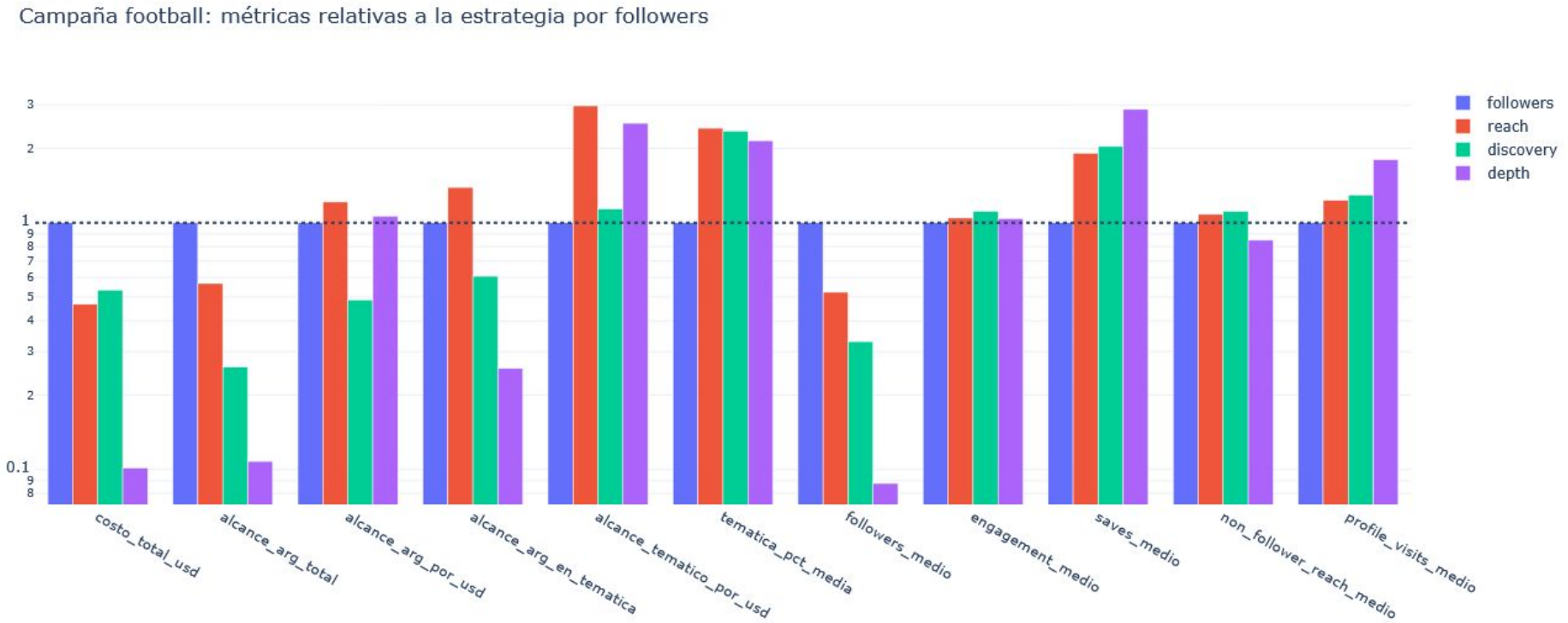


Decisión para las campañas

Estrategia segmentada por temática/audiencia



Estrategia segmentada por temática/audiencia



Estrategia tradicional

Ventajas
- Alta visibilidad inicial

Desventajas
- Audiencias desalineadas
- Bajo alcance en el público objetivo por dólar invertido

Enfoque multivariado

Ventajas
- Mejora la calidad de la interacción
- Ajuste temático
- Llegada al público objetivo

Desventajas
- Requiere un análisis de datos más complejo
- Mantenimiento continuo

Operativa y Mantenimiento

Proyección de Nuevos Creadores

- **PCA / Regla de Oja**: Estandarización del nuevo perfil y proyección directa sobre el vector de pesos convergido
- **Red de Kohonen**: Cálculo de distancia e identificación de la Neurona Ganadora (BMU) para asignación espacial inmediata

Reentrenamiento Periódico y Variaciones Estacionales

- **Métricas mensuales**
- **Adaptación al mercado:** Actualización iterativa de los pesos sinápticos para reflejar dinámicas y tendencias estacionales.

Gestión del Pipeline de Datos

- **Nuevas variables:** Reentrenamiento completo
- **Consistencia:** Automatización del pipeline de estandarización para preservar la validez de los espacios proyectados.

EJ2. Memoria asociativa

Hopfield

Se busca encodificar información
DENTRO (de los pesos) de la red

Implementación

Tipos de Ruido

White - Simula desgaste real dónde se desgastan píxeles

Flip - Cambia al valor opuesto del bit

Perlin - Campo aleatorio espacialmente correlacionado (filtro gaussiano multi-escala).

Finger - Mancha conexa/circular centrada en un punto aleatorio (simula dedo/mancha).

```python
def apply_noise(pattern: np.ndarray, level: float, rng: np.random.Generator, noise_type: str = "white", shape: Optional[Tuple[int, int]] = None) -> np.ndarray:
    """Aplica ruido invirtiendo exactamente round(level * N) píxeles del patrón.

    Tipos de ruido soportados:
    - "white": simula desgaste real dónde se destiñen píxeles.
    - "flip": invierte aleatoriamente los pixeles elegidos.
    - "finger": mancha conexa/circular centrada en un punto aleatorio (simula dedo/mancha).
    - "perlin": campo aleatorio espacialmente correlacionado (filtro gaussiano multi-escala).
    """
    noisy = np.asarray(pattern, dtype=np.int8).copy()          # Copia el patrón para no mutar el original
    n = noisy.size                                             # Cantidad total de píxeles (N)
    n_flip = int(round(level * n))                             # Número exacto de píxeles a perturbar
    if n_flip <= 0:                                            # Si el nivel no alcanza a alterar píxeles:
        return noisy                                           # Retorna el patrón sin cambios
    if n_flip >= n:                                            # Si se perturba el 100% del patrón:
        return -noisy                                          # Retorna el patrón invertido

    if noise_type == "white" or shape is None:                 # Ruido blanco (desgaste / fading a fondo -1)
        idx = rng.choice(n, size=n_flip, replace=False)        # Elige píxeles uniformemente al azar
        noisy[idx] = -1                                        # Apaga los píxeles a -1 (fondo blanco)
        return noisy                                           # Retorna el patrón degradado

    if noise_type == "flip":                                   # Ruido de inversión aleatoria de signo
        idx = rng.choice(n, size=n_flip, replace=False)        # Elige píxeles uniformemente al azar
        noisy[idx] *= -1                                       # Invierte la polaridad (+1 <-> -1)
        return noisy                                           # Retorna el patrón ruidoso

    rows, cols = shape                                         # Dimensiones 2D de la imagen (alto, ancho)
    if noise_type == "finger":                                 # Ruido localizado tipo mancha o huella
        r0 = rng.uniform(0, rows - 1)                          # Centro aleatorio en el eje vertical
        c0 = rng.uniform(0, cols - 1)                          # Centro aleatorio en el eje horizontal
        rr, cc = np.indices((rows, cols))                      # Malla de coordenadas espaciales 2D
        dist = np.sqrt((rr - r0) ** 2 + (cc - c0) ** 2)        # Distancia euclídea radial al centro
        dist += rng.normal(0, 0.35, size=(rows, cols))         # Añade rugosidad estocástica al borde
        idx = np.argsort(dist.ravel())[:n_flip]                # Selecciona los píxeles más cercanos al foco
        noisy[idx] = -1                                        # Apaga la región manchada a -1
        return noisy                                           # Retorna el patrón manchado

    if noise_type == "perlin":                                 # Ruido espacialmente correlacionado (textura)
        field = np.zeros((rows, cols), dtype=np.float64)       # Campo escalar 2D acumulador
        for sigma in (1.0, 2.0, 4.0):                          # Filtro gaussiano a múltiples escalas
            raw = rng.standard_normal((rows, cols))            # Muestra aleatoria normal por píxel
            field += gaussian_filter(raw, sigma=sigma)         # Acumula componentes suaves y amplias
        idx = np.argsort(field.ravel())[:n_flip]               # Selecciona las zonas de mayor intensidad
        noisy[idx] *= -1                                       # Invierte los píxeles en esas regiones
        return noisy                                           # Retorna el patrón con textura suave

    raise ValueError(f"Unknown noise_type: {noise_type!r}")    # Error si el tipo de ruido no existe
```

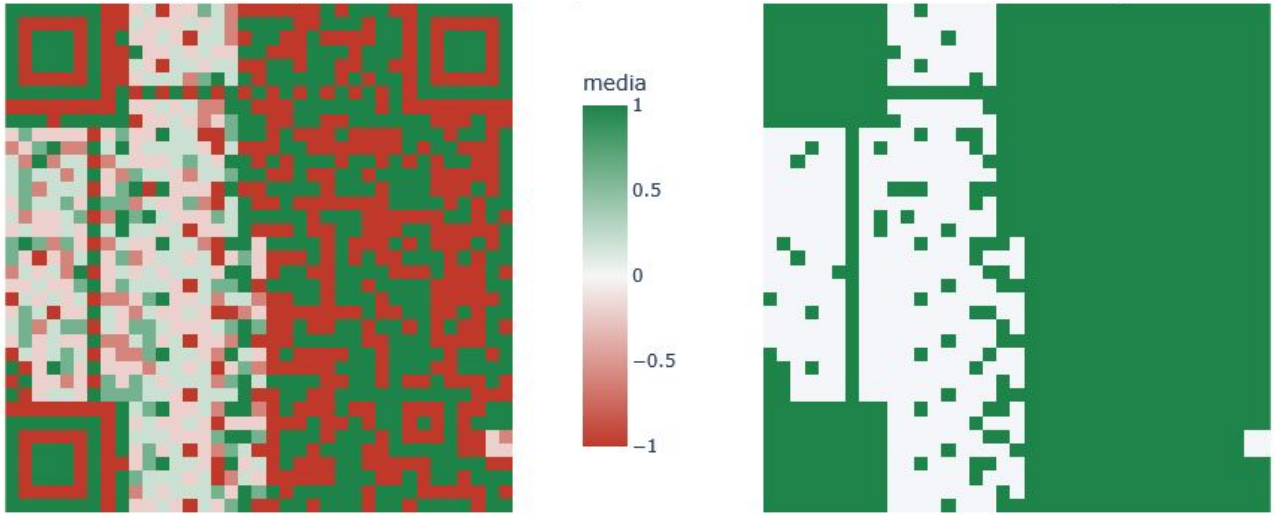
Análisis de datos



QRS a analizar

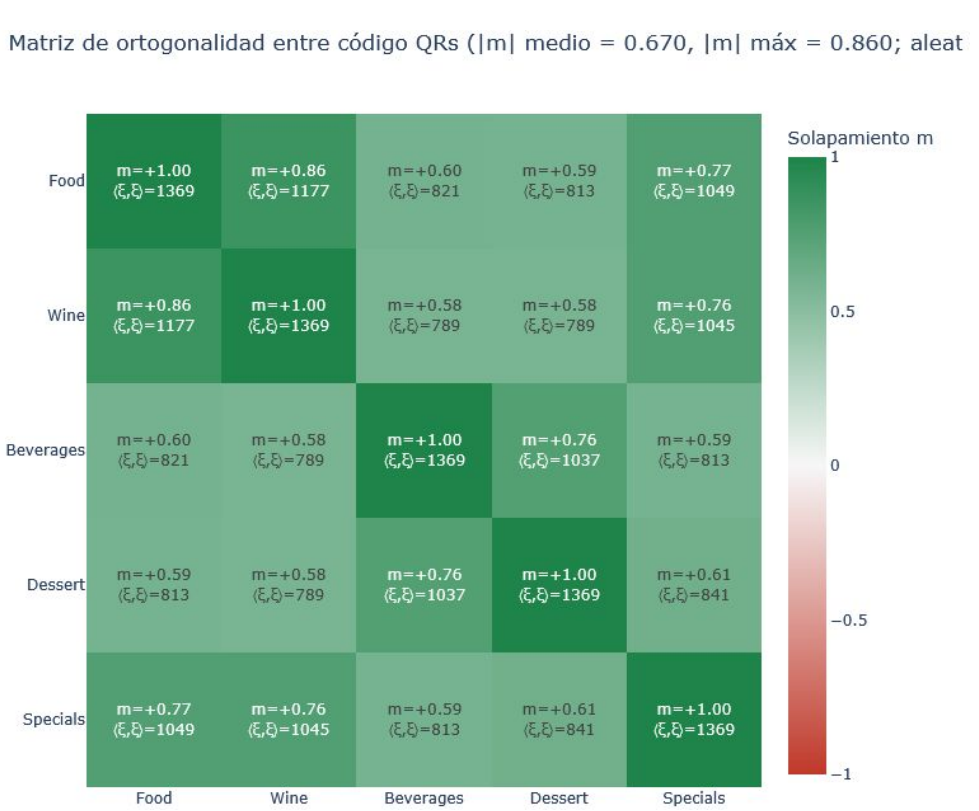
Lo que cambia (o no) en los QR

¡70% de similitud global!



- La parte marcada en verde es la que se repite en todos los QR.
- Quedándonos solo con la parte blanca, que es la que cambia, se puede aumentar la ortogonalidad y mejorar los resultados de Hopfield.

Ortogonalidad



Capacidad de almacenamiento



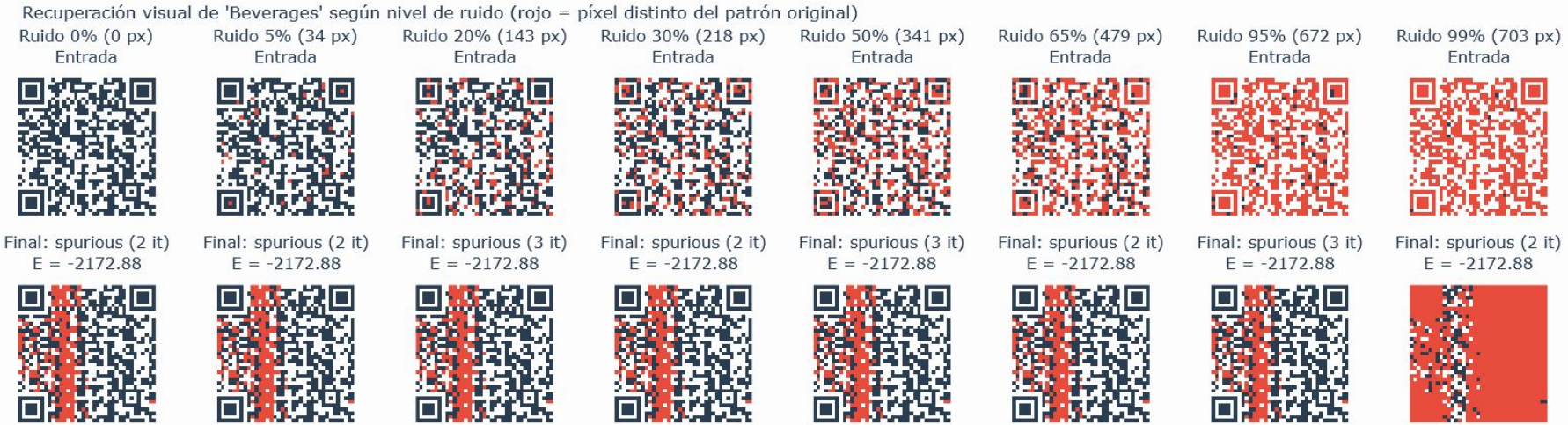


Secuencia óptima (máx. estabilidad)

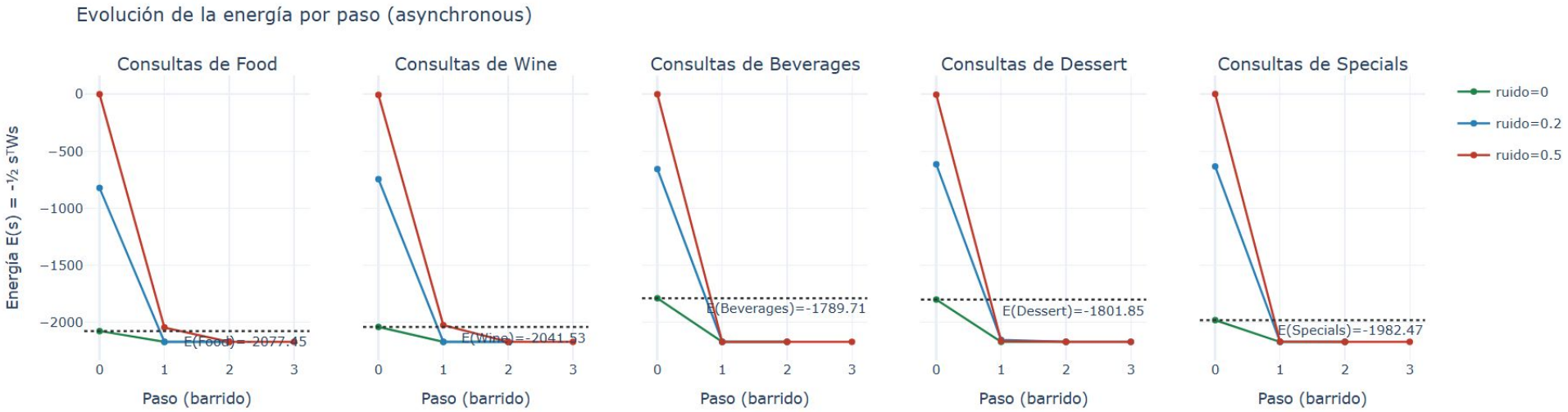
Óptimo a sacar: Wine

Resultados

Hebb: No aprende

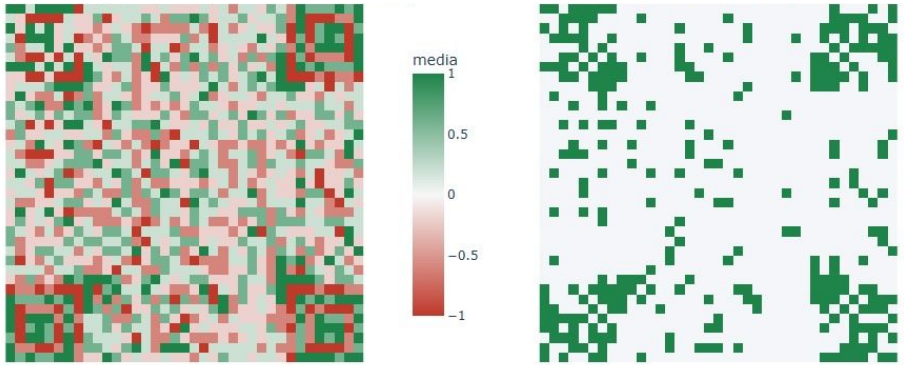


Estudio de energía



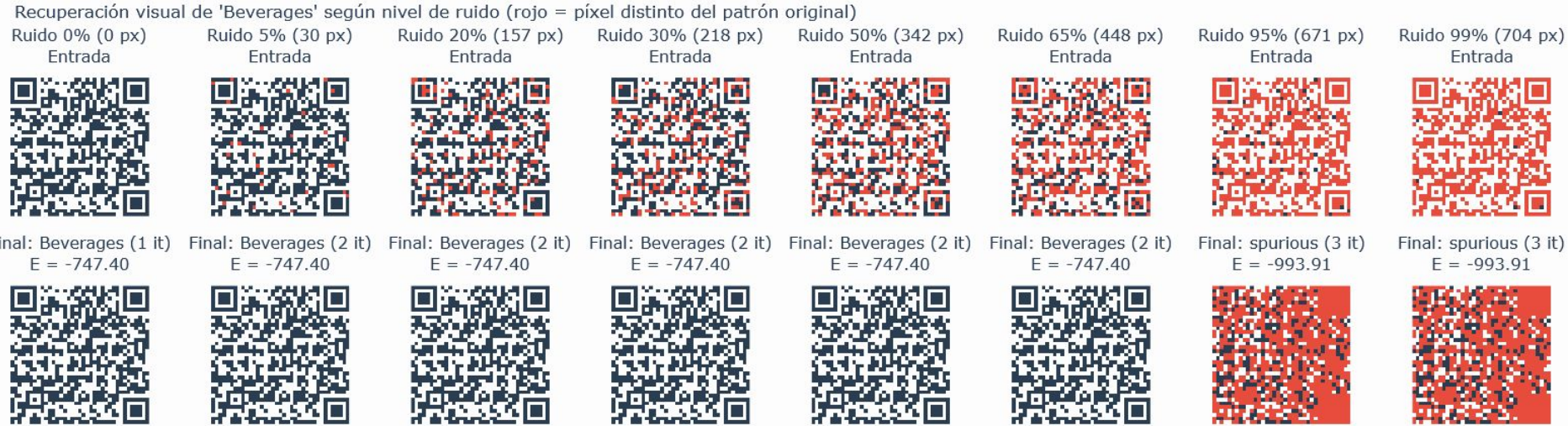
Optimización 1: Rotar los QRS





Rotar los QRS

Sólo diferencia los que tienen distinta rotación



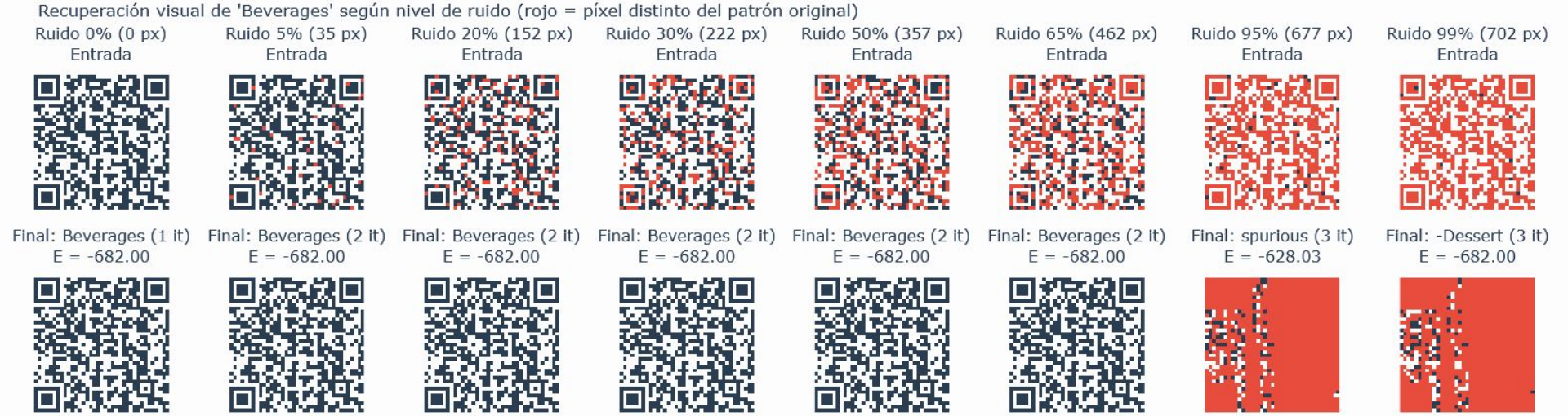
Optimización 2: “Máscara de bits” + Rotación
(¡Logra predecir!)



OBS: SÓLO “Bitmask” **SIN** Wine predice correctamente



Optimización 3: Regla de aprendizaje pseudoinversa



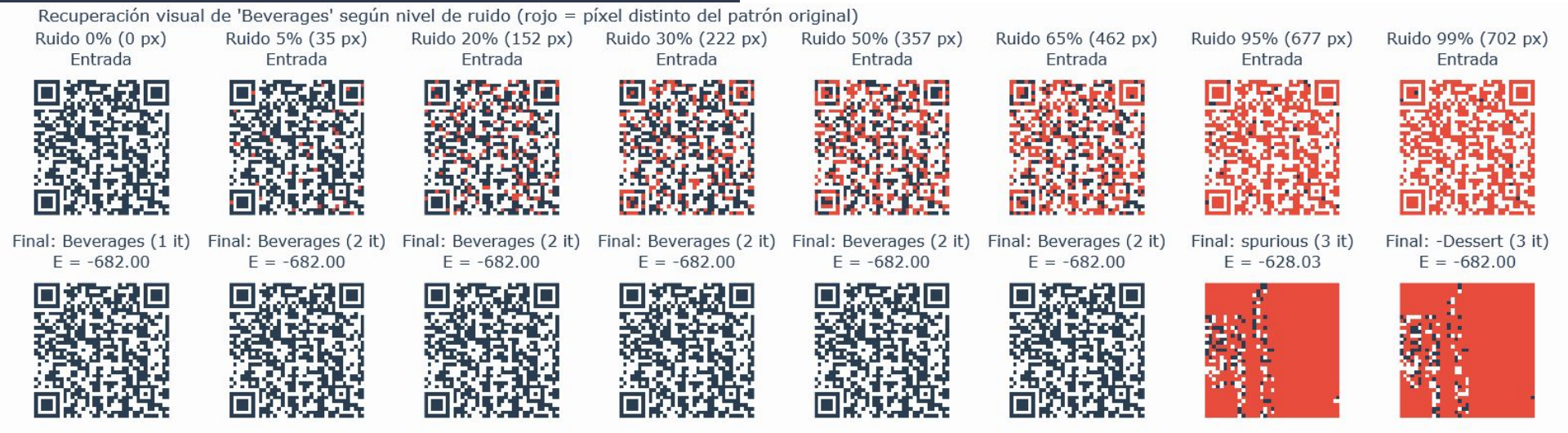
Optimización 3: Regla de aprendizaje pseudoinversa

Ventajas:

- Sin crosstalk
- Mayor capacidad: P<=N, siempre que sean L.I

Desventajas:

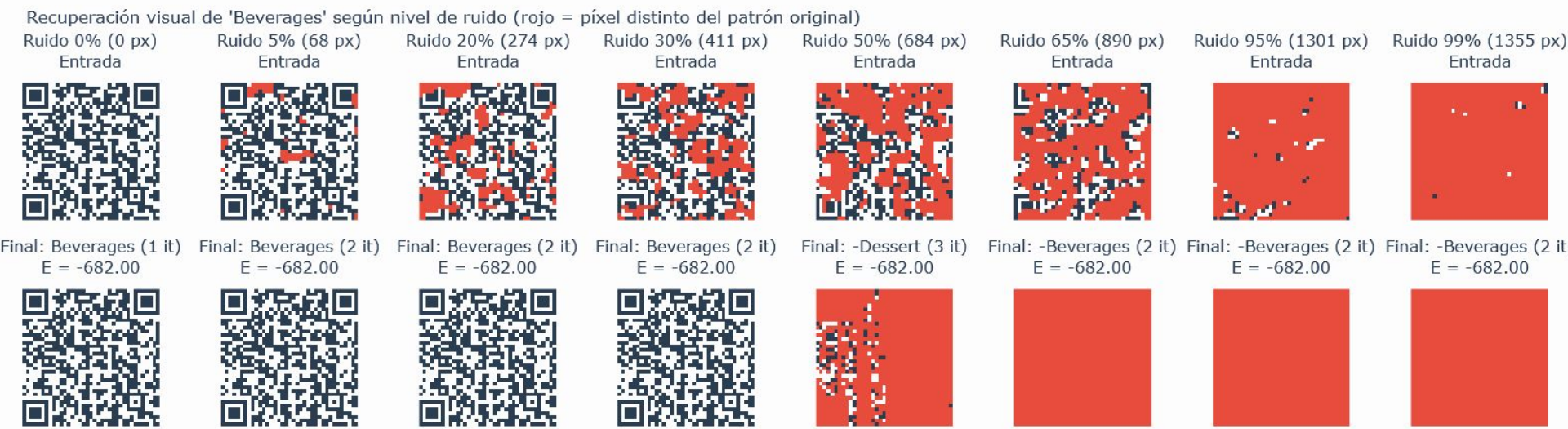
- Computacionalmente costoso
  - (O(P*N²) vs O(N³))



$$W = X^{\dagger}X$$



Pseudoinversa + Finger / Pseudoinversa + Flip



Pseudoinversa + Perlin



Pseudoinversa + Perlin